

How to Build a Good Practice Software Project Portfolio?

Hennie Huijgens
Delft University of Technology and
Goverdson, Delft, The Netherlands
h.k.m.huijgens@tudelft.nl

Rini van Solingen
Delft University of Technology and
Prowareness, Delft, The Netherlands
d.m.vansolingen@tudelft.nl

Arie van Deursen
Delft University of Technology
Delft, The Netherlands
arie.vandeursen@tudelft.nl

ABSTRACT

What can we learn from historic data that is collected in three software companies that on a daily basis had to cope with highly complex project portfolios? In this paper we analyze a large dataset, containing 352 finalized software engineering projects, with the goal to discover what factors affect software project performance, and what actions can be taken to increase project performance when building a software project portfolio. The software projects were classified in four quadrants of a Cost/Duration matrix: analysis was performed on factors that were strongly related to two of those quadrants, Good Practices and Bad Practices. A ranking was performed on the factors based on statistical significance. The paper results in an inventory of ‘what factors should be embraced when building a project portfolio?’ (Success Factors), and ‘what factors should be avoided when doing so?’ (Failure Factors). The major contribution of this paper is that it analyzes characteristics of best performers and worst performers in the dataset of software projects, resulting in 7 Success Factors (a.o. steady heartbeat, a fixed, experienced team, agile (Scrum), and release-based), and 9 Failure Factors (a.o. once-only project, dependencies with other systems, technology driven, and rules- and regulations driven).

Categories and Subject Descriptors

D.2.8 [Software Engineering]: Metrics – *performance measures, process metrics, product metrics.*

General Terms

Measurement, Economics.

Keywords

Success Factor, Failure Factor, Good Practice, Bad Practice, Productivity, Time-to-market, Quality, Agile, Learning Cycle.

1. MOTIVATION

1.1 Problem Statement

Growing complexity faces many software engineering companies nowadays with a portfolio- and project control capability that is lagging behind with their high IT-expenditure. A trend towards rapid application development, acceleration of the pace of change in information technology, in organizations, in competitive countermeasures, and in the environment has caused increasing frustration with heavyweight plans [1]. An answer to this challenge can be found in a need for instruments and techniques that support

transparency and orchestration of software engineering activities, showing organizations what they can learn from their best performing projects (good practices), and from their worst performing projects (bad practices). Especially in complex environments successful software engineering requires companies to pay special attention to a learning capability that supports flexible portfolio- and project management to prevent from decreasing productivity, increasing time-to-market, and low software quality [2]. However this problem seems hard to solve. Still many software development organizations have enormous difficulties developing reliable effort estimates that result in on-time and on-budget delivery of their software products [3], [4]. Many companies have no or limited historic reference data on their software engineering activities available [5], [6]. This limits existing solutions on software estimation to immature and unreliable estimation techniques and hinders learning. An often-heard goal of continuous improvement seems searching for the pot of gold at the end of the rainbow.

What can we learn here from historic data that is collected in three software engineering companies that on a daily basis had to cope with such highly complex project portfolios?

1.2 Research Objectives

Within the scope of the implementation of several measurement programs as part of process improvements, a large dataset containing 352 software projects was collected in practice in three different companies during a time-span of six years (2008 to 2013). We define the following research question:

What factors affect software project performance, and what actions can be taken to increase project performance when building a software project portfolio?

1.3 Context

Data was collected on finalized software engineering projects within three different companies. The companies – two large banks (in this paper referred to as ‘B1’ and ‘B2’), and one telecom provider (referred to as ‘T’) – were, with regard to their software engineering activities, comparable to each other. The size of the software project portfolio of the banks was considerably larger than that of the Telco: The measured yearly throughput of software projects for B1 was approximately 10,000 function points (FPs) [7] [8], for B2 this was approx. 8,000 FPs per year, and T measured approx. 2,000 FPs per year. However, on a business domain scale the software engineering activities were equal in many ways. Software engineering was organized in projects or releases, different delivery models were adopted, and software engineering was characterized by a variety of programming languages and business domains (varying from business intelligence systems to mobile apps).

In all three companies a comparable and similar approach for measurement and analysis of software projects was implemented during the data collection period. A so-called *learning-cycle* was implemented: (1) finalized software projects were measured, col-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

ICSE Companion '14, May 31 – June 7, 2014, Hyderabad, India
Copyright 2014 ACM 978-1-4503-2768-8/14/05...\$15.00
<http://dx.doi.org/10.1145/2591062.2591187>

Table 1. Overview of the measurement repository

Category	Type	Occurrence	N	Definition of Project Factors
Company ID (ORG)	Nominal	3	352	Identification code of the company where a project was performed; three companies were applicable (nr. of occurrence between brackets): B1 (206), B2 (125), T (23).
Project ID	Nominal	352	352	Identification code of a project.
Year of Go Live	Ordinal	6	352	Year when a project was finalized; the following years Go Live were applicable: 2008 (32), 2009 (59), 2010 (81), 2011 (131), 2012 (41), 2013 (10).
Business Domain (BD)	Nominal	10	352	Customers business sector; the following BD were applicable: Finance & Risk (54), Internet & Mobile (54), Payments (50), Client & Account Management (incl. CRM systems) (46), Savings & Loans (40), Organization (incl. HRM) (31), Call Centre Solutions (21), Mortgages (21), Data warehouse & BI (18), Front Office Solutions (17).
Primary Programming Language (PPL)	Nominal	21	352	Primary used programming language; the following PPL were applicable: JAVA (154), .NET (59), COBOL (55), ORACLE (29), SQL (9), 3GL (8, unknown was what specific languages were applicable here), Visual Basic (6), RPG (6), FOCUS (5), PowerBuilder (5), PRISMA (4), MAESTRO (3). In the analysis 4 th Generation (1), PL1 (1), JSP (1), C++ (1), Clipper (1), Document (1), PL/SQL (1), Siebel (1) and Package (1, unknown what specific language was applicable) were referred at as Other.
Delivery Model (DM)	Nominal	2	352	Classification of the used delivery model; two DM were applicable: Structured (e.g. Waterfall) (307), and Agile (Scrum) (45). One project reported as DM RUP is included in the analysis of Structured.
Development Class (DC)	Nominal	4	352	Classification of the development; the following DC were applicable: New development (173), Major enhancement (25-75% new) (124), Minor enhancement (5-25% new) (27), Conversion (28).
Project Keyword (KW)	Nominal	20	351	Characteristics on a specific project (multiple keywords could be mapped on one project, on one project no keyword was mapped); the following keywords were applicable: Single-application (270), Business driven (150), Release-based (one application) (144), Once-only project (122), Phased project (part of program) (65), Fixed, experienced team (62),), Technology driven (58), Steady heartbeat (49), Dependencies with other systems (41), Migration (35), Rules & Regulations driven (33), Multi-application release (21), Many team changes, inexperienced team (17), Package with customization (16), Legacy (15), Security (14), Pilot; Proof of Concept (10), Bad relation with external supplier (9), New technology, framework solution (3), Package off-the-shelf (1).
Measure	Type	Occurrence*	N	For every project in the repository the measurements indicated in the table below are inventoried.
Size (FP)	Ratio	-	352	Size of a project in Function Points (FPs).
Duration	Ratio	-	352	Duration of a project in Months; measured from the start of Project Initiation to (technical) Go Live.
Cost	Ratio	-	352	Cost of a project in Euros; measured from the start of Project Initiation to (technical) Go Live.
Effort	Ratio	-	352	Effort spent in a project in Person Hours (PHRs); measured from the start of Project Initiation to (technical) Go Live.
Defects	Ratio	-	172	The number of errors or faults found in a project from System Integration Test to (technical) Go Live. Not for all projects defects were administrated; for 172 projects defects info was recorded in the repository.

*No occurrences are indicated for the 5 Measures, due to the fact that these are different for every measured project.

lected in a measurement repository, and analyzed, (2) data of groups of finalized projects was analyzed on specific trends and benchmarked with internal and external peer-groups, and (3) finally trends were incorporated in an estimation process for newly to be started projects. With regard to step 1 of the learning cycle the measurement repository included both quantitative project data (e.g. size, cost, effort, duration, and defects) and qualitative project data (e.g. reasons named by the project manager that could explain the projects' performance).

In this paper we subsequently discuss research design, execution, analysis, interpretation, related work, and conclusions and future work.

2. RESEARCH DESIGN

2.1 Approach

There may be a large gap between how different stakeholders, and researchers, define success and failure of software projects. Similarly to Lindberg [9], we relate success and failure within this research to better or worse than average cost, effort, and duration performance. Although, where Lindberg compares to industry, we posed to focus the study on identifying success and failure factors of the sample software projects themselves. Every single project from the sample is compared with the average performance of the whole repository. The idea behind this is that a focus at identifying projects that performed better than average, and projects that

performed worse than average, might help companies to realize their ultimate goal of continuous improvement. The refined research objectives of this study are to identify factors that are instrumental in software project success or failure. Following from that, we analyze actions that help to improve the performance of such projects. In the scope of this study a performance-rating ‘better than others’ must be read as better than the average performance of the whole sample. ‘Worse than others’ must be read as worse than the average performance of the whole sample.

2.2 Design

The strategy for this research can be defined as a data analysis project. We did not study individual projects in our sample, but the primary author collected and maintained the majority of the projects in the measurement repository. A minority of the projects was collected by third parties. Our goal was to explore the performance of finalized software projects and identify key success and failure factors in order to build a Good Practice Project Portfolio. Our study proposition was twofold. First, we hypothesized that within the repository good performers and bad performers can be distinguished in a quantitative way. Second, we expected that the qualitative information in the repository gives valuable, additional information on success and failure of the projects and on software engineering projects in general.

2.3 The Measurement Repository

Table 1 gives an overview of the measurement repository, including the different aspects that are analyzed in our research. All data in the repository was collected before we started our research, and only with practical analysis purposes in mind. All data is about finalized projects: the dataset does not hold any data on prematurely stopped or failed projects. All projects are related to solution delivery, i.e. an information system was modified or completely new designed. The dataset contains software engineering projects, which in some cases contain an infrastructure component or an implementation of middleware: no full infrastructure or middleware projects are included in the repository. With regard to our research, the dataset is where possible and applicable, discussed with the measurement team that was responsible for the collection. For the research paper all available data in the repository has been used; no deviations or outliers have been removed from the dataset.

The repository contains both quantitative data (e.g. ratios on size, duration, and cost), and qualitative data (e.g. applicable keywords). The repository holds data of 352 software engineering projects carried out in three companies during a period from 2008 to 2013. The projects represent a total amount spent of 266M Euro. Project cost range from 12K Euro to 6.8M Euro. The sum of function points across projects in the repository is 91,105 FPs; ranging from projects with a size of 5 FPs to 4,600 FPs. The project duration ranges from 0.9 Months to 26.8 Months.

Qualitative data is recorded for the following research categories:

- Business Domain (BD);
- Primary Programming Language (PPL);
- Company (ORG);
- Delivery Model (DM);
- Development Class (DC);
- Size Category (SC), and;
- Project Keyword (PK).

Within these 7 research categories are in total 56 project factors inventoried in the repository (see Table 1 for an overview).

For all inventoried projects size is measured in Function Points (FPs). Function Point Analysis (FPA) has been performed either by an expert member of a measurement team or an external certified FPA-specialist, according to ISO-standardized functional size measurement (FSM) methods [8] [7]. FPA was performed based on sets of final project documentation that usually were delivered by the project manager. All projects were collected and analyzed according to the so-called SEI Core Metrics, a standard set of process-based software engineering metrics [10] [11]. Project duration was measured in a number of months from the start of the project initiation to technical Go Live. Effort was measured in hours, although, measurement specialists that were responsible for the data collection reported that data quality with regard to effort was low, especially where (globally distributed) external suppliers were involved in a project. Because of that we decided to use both effort and cost data for analyzing purposes. Project cost – in this case all project related cost excluding investments (e.g. software license costs) – were recorded in Euros in the measurement repository. Besides that for a limited set of projects (N = 172) the number of defects was recorded.

2.4 Analysis Procedure

All software projects in the measurement repository are analyzed from a portfolio point of view, meaning that we were interested in particular in the mutual coherence between the measured projects. We perform the analysis of the portfolio (the measurement repository) in four steps:

1. First, we analyze the overall average performance of all projects in the repository with regard to project size, project cost, and project duration, measured in function points, euros, and months, respectively.
2. Subsequently, we analyze what projects performed as a good practice, and what projects did perform as a bad practice. In the context of our research a project is assessed as a good practice when the performance on both cost and duration is better than the average cost and duration corrected for the applicable project size. A project is assessed as a bad practice when the performance on both cost and duration is worse than the average cost and duration again correct for the project size. To do so we classify the projects in a Cost/Duration Matrix (see Figure 2). We challenge the project performance exclusively against the performance of the whole sample (internal benchmarking). We compare the outcome of our analysis with other studies in Section 6 on Related Work.
3. Once all projects are classified in the Cost/Duration Matrix, we analyze how the 56 project factors (as defined Table 1) are related to the four quadrants of the matrix. This analysis results in a percentage based on number of projects per quadrant for every project factor and a percentage based on cost per quadrant for every factor. We assess the outcome of the analysis by calculating the significance of both outcomes (a range of percentages per quadrant based on number of projects, versus a range of percentages per quadrant based on project cost), by performing a chi-square test. An example: for the Primary programming Language (PPL) ORACLE 29 projects are measured, of which 26 small releases (measured in cost and size) score as a Good Practice. Although, when assessed based on project cost it shows that 3 large projects (in size and cost) score as a Bad Practice. This leads to an indistinct result that from a number of project point of view PPL ORACLE scores high in Good Practice, and from a cost point of view it scores

high in Bad Practice. In the further analysis factors with these kinds of indistinct outcomes will be excluded.

4. We identify factors that are strongly related for the composition of software engineering project portfolios, by analyzing specific subsets per research aspect. In other words: ‘What factors should be embraced when composing a project portfolio?’ and ‘What factors should be avoided when doing so?’ For this analysis we define a research aspect to be ‘strongly related’ when the percentage Good Practice or Bad Practice was 50% or more.

Once an inventory of strongly related factors is finalized we test for each of them whether it indeed affects the probability that a project ends up as a Good or Bad Practice. To that end, for each strongly related factor F , we generate a null and alternative hypothesis:

- $H(F, 0)$: Factor F does not influence good (or bad) practice;
- $H(F, 1)$: Factor F influences good (or bad) practice.

To test these hypotheses, we use the binomial distribution to estimate the chance that our actual observation takes place under the null hypothesis, as follows.

Let n be the total number of projects in the repository. Let $prob(F, n)$ be the proportion of projects for which F holds in n , and let k be the number of good (bad) practice projects in which F holds. Then p is given by the binomial distribution as follows:

$$p = (n \text{ over } k) * prob(F, n)^k (1 - prob(F, n))^{(n-k)}$$

We reject the null hypothesis in favor of the alternative hypothesis if the significance p is below 0.05.

3. EXECUTION

3.1 Distribution of the Sample

The distribution of the dataset is described as a positively skewed distribution, or one whose elongated tail extends to the right end of the range. The mean duration in the sample is 8.7 Months, while the median duration is 8.0 Months (min 0.9 Months, max 26.8 Months). The mean project cost is EUR 750,777, while the median project cost is EUR 459,150 (min EUR 12,233, max EUR 6,802,466). The mean project size is 259 FPs; the median project size is 147 FPs (min 5 FPs, max 4,600 FPs).

Although due to the positively skewed distribution analysis based on the median (instead of the mean) might be preferable (the expectation is that outliers will interfere the outcome less), we use the mean in our analysis of factors. After performing both analyses we found, except for some small differences in numbers and percentages, no differences in the outcome of the study (Success and Failure Factors). We assume the Central Limit Theorem to be applicable in this case.

4. ANALYSIS

4.1 Overall Performance Analysis

We analyze the overall weighted average performance, with project size (FPs) as weighting factor, of all projects in the repository with regard to project size, project cost, and project duration. For this purpose we calculate three Performance Indicators:

1. Productivity (PROD); expressed in cost per size unit (EUR/FP) and size unit per hour (FP/HR);
2. Time-to-Market (TTM); expressed in project calendar days per size unit (Days/FP);

Table 2. Average Performance per Size Category.

Performance Indicator	SP	SMP	LMP	LP	Overall
PROD (EUR/FP)	4,364	3,395	2,508	2,111	2,929
PROD (FP/HR)	0.024	0.034	0.045	0.047	0.037
TTM (Days/FP)	2.74	1.03	0.75	0.38	1.08
PQ (Defects/FP)	0.20	0.13	0.13	0.21	0.18
Percentage in Sample	62%	19%	10%	9%	100%
Explanation of abbreviations: SP = Small Projects (<200 FP); SMP = Small Medium Projects (201-400 FP); LMP = Large Medium Projects (401-600 FP); LP = Large Projects (>601 FP); PROD = Productivity in resp. Euros per FP and FP per hour; TTM = Time-to-Market in Days per FP; PQ = Process Quality in Defects/ FP. The indicators are calculated as weighted average, with Size as weighting factor (e.g. Cost (Euros) divided by Size (FP), instead of number of projects as weighting factor.					

3. Process Quality (PQ); expressed in quality of the process per size unit (Defects/FP).

In Table 2 the values of these indicators are inventoried. In order to get more insight in the effects of economy of scale, we divided the sample in four size categories. The used bins, size categories of 200 FP each, all projects larger than 600 FPs are considered large, are commonly used as a measure for project size in two of the applicable companies. Table 2 gives an overview of the weighted average scores per size category within the measurement repository with regard to the Performance Indicators. As the table shows economy of scale does play an important role here: the performance of the projects in the repository, measured in Productivity, Time-to-Market, and Process Quality, is related to the size of a project. The larger the project; the better the performance on average is in terms of time, money, and quality. The table further shows that most projects in the repository could be categorized as small projects (62% of the projects are smaller than 200 FP).

The analysis shows a remarkable fact: while on the one hand medium sized projects show the best performance in terms of time, money, and quality, on the other hand companies give preference to build their portfolio on small or small medium sized projects. An often witnessed adage that ‘small projects do not fail while large projects often do’ might play a role here. An interesting side-effect of the observation is that nowadays software companies, in an attempt to become more agile, tend to opt for small releases (however this is not always the case: our sample holds two large Scrum releases of resp. 1,067 FPs and 4,600 FPs). Yet maybe active steering on economy of scale can be an equally effective – or even more effective – improvement strategy for software companies. We did not study this side-effect; however future research on the background of economy of scale versus agile development methods might help software companies to find an optimum on project size when building a portfolio.

4.2 The Cost/Duration Matrix

As a second step in the analysis all projects from the repository are classified in the four quadrants of a Cost/Duration Matrix, by combining two Plotter Charts (see Figure 1):

1. A chart (left in Figure 1) where all projects from the repository ($N = 352$) are plotted in Size (FP) versus Duration (Months). This plotter chart indicates what projects score below the average trend line ($M = 7.995$, $SD = 5.089$, $r^2 = 0.21$) with regard to project duration, meaning the project duration

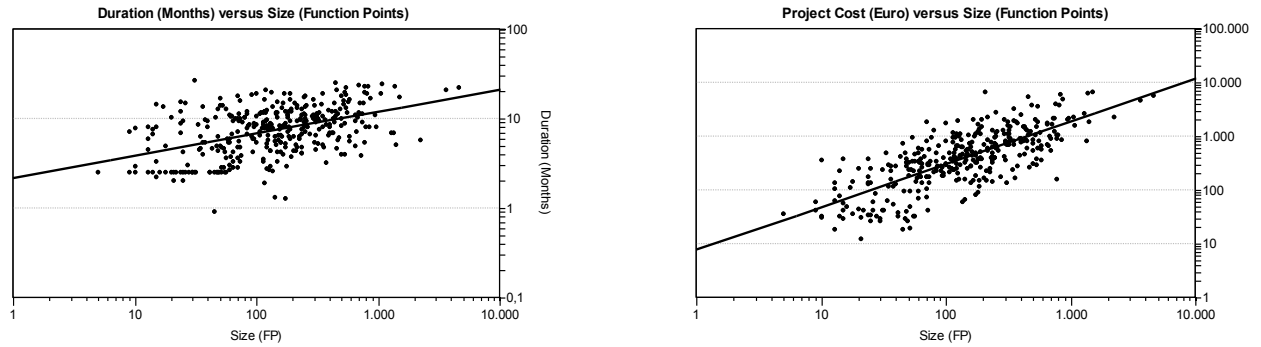


Figure 1. Two plotter charts representing Size (FP) versus Duration (Months) and Size (FP) versus Project Cost (Euros).

is shorter than average, and what projects score above the average trend line, meaning the project duration is longer than average.

2. A chart (right in Figure 1) where all projects from the repository ($N = 352$) are plotted in Size (FP) versus Cost (Euros). This plotter chart indicates what projects score below the average trend line ($M = 750777$, $SD = 1019949$, $r^2 = 0.56$) with regard to project cost, meaning the project cost are less than average, and what projects score above the average trend line, meaning the project cost are higher than average.

For each project the measure of deviation from the average trend line is calculated and expressed in a percentage; negative when below the average trend line, positive when above the trend line. Based on this percentage all projects from the repository are plotted in a matrix, resulting in four quadrants. Each quadrant is characterized by the measure of negative or positive deviation from the average trend (see Figure 2).

Table 3 gives an inventory of the most important performance indicators with regard to the four Cost/Duration Quadrants. The table clearly indicates that projects that score as Good Practice on average show the best Productivity, Time-to-Market, and Process Quality of all four quadrants. And for Bad Practices these performance indicators are the lowest of all four quadrants. The cost of a FP is for example for a Bad Practice as much as four times higher than the average cost for a FP of a Good Practice.

4.3 Analysis of Factors

Once all projects have been classified in the Cost/Duration Matrix, we analyze how the 56 project factors are related to the four quadrants of this matrix (respective Good Practice, Bad Practice, Cost over Time, and Time over Cost). Two different perspectives are analyzed: the distribution over the four quadrants

per number of projects and the distribution over the four quadrants based on project cost. In order to find significant differences between the two perspectives (i.e. number of projects and project cost) we use the chi-square test. After removal of non-significant results we base the interpretation at the percentage number of projects per Cost/Duration Quadrant; resulting in an average percentage per quadrant for all 56 research aspects. A summary of the total analysis, including the result of the chi-square test, is established in the appendix of the accompanying technical report [12].

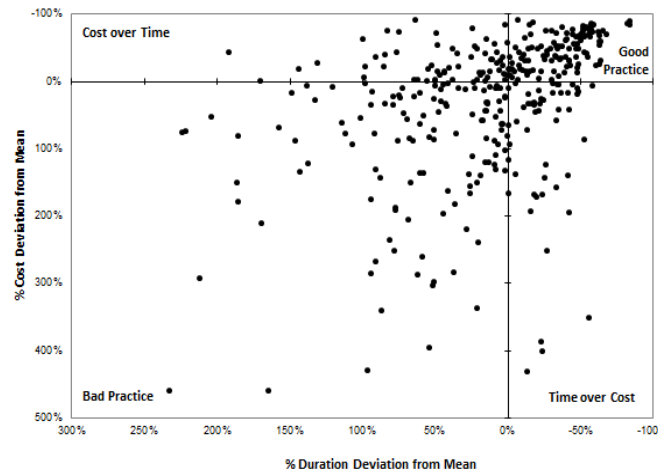


Figure 2. Cost/Duration Matrix.

Explanation of the four quadrants in the Cost/Duration Matrix above:

Good Practice (GP) (upper right): This quadrant shows projects that scored better than the average of the total repository for both cost and duration.

Cost over Time (CoT) (upper left): In this quadrant projects are reported that scored better than the average of the total repository for cost, yet worse than average for duration.

Time over Cost (ToC) (bottom right): In this quadrant projects are plotted that scored better than the average of the total repository for duration, however worse than average for project cost.

Bad Practice (BP) (bottom left): This quadrant holds projects that scored worse than the average of the total repository for both cost and duration.

Table 3. Average Performance per Cost/Duration Quadrant.

Performance Indicator	GP	CoT	ToC	BP
PROD (EUR/FP)	1,285	1,448	3,834	5,285
PROD (FP/HR)	0.082	0.064	0.028	0.021
TTM (Days/FP)	0.64	0.92	0.75	1.77
PQ (Defects/FP)	0.06	0.20	0.19	0.27
Number in Sample	114	55	51	131

4.4 Success and Failure Factors

As a fourth and last step we identify specific success factors and failure factors for the composition of software engineering project portfolios, by analyzing how the scores of the different research aspects relate to the Cost/Duration Quadrants. We identify what aspects are strongly related (50% or more) to a high percentage of Good Practice and what aspects are strongly related (50% or more) to a high percentage of Bad Practice.

5. EVALUATION

In this section we evaluate results and implications of the study. Analysis results in an inventory of factors that are strongly related to a high percentage of Good Practice (also referred at as Success Factors) and factors that are strongly related to a high percentage of Bad Practice (Failure Factors). To create better insight in the measure of modification of the factors, both Success and Failure Factors are classified into categories of factors that are IT-organizational, business-organizational, and factors that are primarily technical. In total 10 research aspects were found to be strongly related to Good Practice (Success Factors), and 13 research aspects strongly related to Bad Practice (Failure Factors). Besides that we found 1 factor that is strongly related to a high percentage of Cost over Time, and 2 factors that could be related to a high percentage of Time over Cost.

5.1 Factors Excluded from the Inventory

In order to find significant differences between the two perspectives (i.e. number of projects and project cost) we use the chi-square test. We have found a number of significant differences between the two perspectives: 8 out of 56 factors are excluded from the interpretation. One factor, KW Package off-the-shelf (without customization), scores as related to a high percentage of Good Practice; however only one such project was in the sample. With regard to 6 factors we found a low significance between the percentages GP, CoT, ToC, and BP, measured on number of projects and the percentages measured on project cost (χ^2 of lower than 5). These 6 excluded factors are:

- PPL Oracle, $\chi^2(1, N = 29) = 0.38, p = .00$.
- BD Finance & Risk, $\chi^2(1, N = 54) = 3.27, p = .00$.
- PPL RPG, $\chi^2(1, N = 6) = 2.68, p = .18$.
- KW Phased project, $\chi^2(1, N = 65) = 4.26, p = .05$.
- DC Minor Enhancement, $\chi^2(1, N = 27) = 2.78, p = .04$.
- PPL 3GL, $\chi^2(1, N = 8) = 3.18, p = .19$.

Two of these factors, PPL ORACLE (86% GP based on number of projects) and BD Finance & Risk (69% GP based on number of projects) both score as factors that are strongly related to a high percentage of Good Practice, for which a remark is in place. PPL ORACLE scores with a relation to a high percentage of Good Practice when looked upon from number of projects (86%), however it also has a high percentage of Bad Practice when looked upon from project cost (73%). This effect is caused by the fact that a sample of 29 PPL ORACLE projects is analysed, including 26 very well performing small releases belonging to two applications [13]. The other three medium sized PPL ORACLE projects did not perform well; two score as a Bad Practice, and one scores in the Time over Cost quadrant, resulting in a low χ^2 -score. This also affects the score for BD Finance & Risk, as all 26 good performing projects are performed within this domain, including one medium sized badly performing project that influences the score with regard to project cost, again leading to a low χ^2 -value.

Finally a factor that is excluded from the inventory is the category PPL Other: representing a collection of 9 primary programming languages that were recorded for only one project each in the repository.

5.2 Factors Strongly Related to Good Practice

We identified 10 factors that were strongly related to a high percentage of Good Practice:

Table 4. Overview of factors strongly related to Good Practice.

Project Factor	% GP	N	p
PPL Visual Basic	83	6	.03
PPL FOCUS	80	5	.06
KW Steady heartbeat	71	49	.00
KW Fixed, experienced team	66	62	.00
BD Data Warehouse & BI	61	18	.02
PPL PowerBuilder	60	5	.14
DM Agile (Scrum)	56	45	.00
PPL SQL	56	9	.10
BD Organization	52	31	.02
KW Release-based (one application)	50	144	.00

5.3 Success Factors for Software Projects

After testing for statistical significance we found that of those 10 factors 7 satisfied the alternative hypothesis (H_1) (see Figure 3). 4 factors of these 7 are tested to be strongly significant ($p < 0.01$) to influencing project success (high probability to end up as a Good Practice), and due to that can be specified as strongly significant Success Factors for Software Projects (ranking based on probability of success):

1. Steady Heartbeat;
2. Fixed, experienced team;
3. Agile (Scrum);
4. Release-based (one application).

It is likely that these four strongly significant Success Factors are related to each other, since release-based working, a steady heartbeat, and a fixed, experienced team, are in fact preconditions of an agile (Scrum) way of working. However keep in mind that not all projects where factor 1, 2, or 4 was applicable used an agile delivery model. In fact all types of projects can adopt these Success Factors, without opting for a specific delivery model. The promising idea behind these four Success Factors is that they all can be implemented relatively easy and fast in any software engineering company: no big organizational or technical changes are needed to start working according to these Success Factors. In a way an organization can decide to start working this way by tomorrow.

Besides these four strongly significant Success Factors, we found out of the group of 7 factors that were strongly related to Good Practice, 3 factors with a significant probability ($p < 0.05$) to perform as a Good Practice (ranking based on probability):

5. Business Domain Data Warehouse & BI;
6. Business Domain Organization;
7. Programming Language Visual Basic.

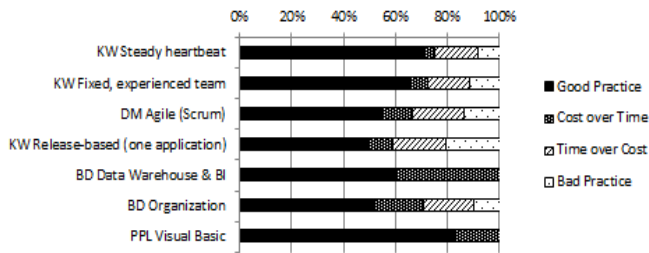


Figure 3. Overview of Success Factors.

One could argue that a difference is applicable with regard to the three Success Factors above with the Success Factors 1 to 4 on the fact that (where 1 to 4 seems relatively easy to implement) 5 to 7 are more difficult to change. A software company cannot simply change its business organization structure overnight, and opting for another programming language asks for a more long term approach.

As an example we do see implementations in practice where a company actively builds its application portfolio on a limited number of programming languages, however in many cases the reason behind such an improvement strategy lies in steering on shortage versus availability of language-specific developers (avoiding of high labor cost) instead of implementing continuous performance improvement.

Although we found a strong significance for PPL Visual Basic a remark is in place: 6 projects were analyzed of which 5 scored as Good Practice and one as Cost over Time. The projects took place in two different companies and varied in size from 27 to 586 FPs.

5.4 Factors Strongly Related to Bad Practice

We identified 13 factors that were strongly related to a high percentage of Bad Practice:

Table 5. Overview of factors strongly related to Bad Practice.

Project Factor	% BP	N	p
KW Security	71	14	.02
KW Many team changes, inexperienced team	71	17	.01
BD Mortgages	67	21	.01
KW Bad relation with external supplier	67	9	.07
KW New technology, Framework solution	67	3	.21
KW Rules & Regulations driven	64	33	.01
KW Dependencies with other systems	63	41	.00
KW Legacy	60	15	.05
KW Migration	57	35	.02
KW Technology driven	57	58	.00
BD Client & Account Management	54	46	.01
KW Once-only project	52	122	.00
KW Pilot, Proof of Concept	50	10	.15

5.5 Failure Factors for Software Projects

After testing for statistical significance we found that of those 13 factors 9 satisfied the alternative hypothesis (H_1) (see Figure 4). 4

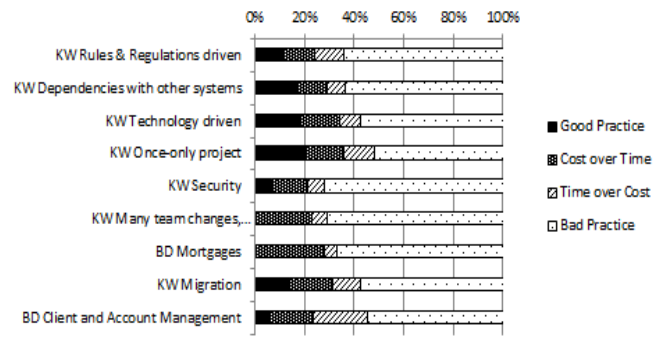


Figure 4. Overview of Failure Factors.

factors of these 9 are tested to be strongly significant ($p < 0.01$) to influencing project failure (high probability to end up as a Bad Practice), and due to that can be specified as strongly significant Failure Factors for Software Projects (ranking based on probability of failure):

1. Rules & Regulations driven;
2. Dependencies with other systems;
3. Technology driven;
4. Once-only project.

An interesting relation can be assumed here between the strongly significant Success Factor Release-based, and the strongly significant Failure Factor Once-only project. Although our research does not show, we assume that starting up a once-only project, including having to do things for the first time, and for one project only, leads to a high probability of ending in between Bad Practice. On the other hand the repeating character of release-based working, including the effect of learning on-the-job and creating an experienced team, creates a high probability to end up as a Good Practice.

Next to these four strongly significant Failure Factors, we found that 5 factors from the group of strongly related to a high percentage of Bad Practice showed a significant ($p < 0.05$) probability to perform as a Bad Practice (ranking based on probability of failure):

5. Security;
6. Many team changes, inexperienced team;
7. Business Domain Mortgages;
8. Migration;
9. Business Domain Client & Account Management.

5.6 Factors Related to CoT and ToC

We did not find any factors that were strongly related to a high percentage of Cost over Time (cost lower than average, duration longer than average) or Time over Cost (cost higher than average, duration shorter than average) that were significant ($p < 0.05$) for respectively CoT or ToC.

6. DISCUSSION

In this section we discuss the most important limitations with regard to the study.

6.1 Research on an Existing Repository

Owing to the fact that the data of software engineering projects that is used for the research was collected in a practical setting and primary for practical purposes, this paper must emphatically been seen as the result of analysis that was performed on an existing

measurement repository. As a consequence of that we had to cope with the data that was available: no additional data was to be collected at a later stage. The dataset only contained data of finalized (successful) projects; no data of prematurely stopped or failed projects was available. In particular for the identification of Bad Practices, the study of failing projects will be relevant.

6.2 Uncertainties Related to Software Metrics

A remark is in place with regard to uncertainty ranges and data collection problems that exist in repositories of software metrics data. Variation ranges of about $\pm 15\%$ between projects and organizations due to the counting rules for data [14]. Data collection problems such as missing values, cost segmentation due to different counting methods, too many factor levels, and not enough factors [15]. Algorithmic estimation models based on historic project data (e.g. SLIM, Function Points, COCOMO) do not model the factors affecting productivity very well [16]. All are examples of uncertainties that might apply to our research data too, in spite of a great effort that was put in quality assurance activities during data collection by the teams that helped building our research repository.

However, a majority of the mentioned problems are related to estimation based on algorithmic models derived from historic data, while this is not the case in our research. We divided our research dataset in relatively big parts (the four Cost/Duration quadrants) and applied impact factors on two of these. The uncertainties that do apply on algorithmic-based effort and schedule estimation are not automatically applicable to our research too.

Looking at the research challenge raised by Boehm and Sullivan [14] that “we need to learn how to think about and manage software development as an investment activity, the goal of which is to create maximum value for the resources invested” we feel that (unless any data collection problems and uncertainties) the goal of our study to help enterprises build good practice project portfolios is a step ahead towards solving that challenge.

6.3 Business Domain and Language

Due to the upfront collection of project data, we could not exactly identify the cause behind the fact that programming language Visual Basic and the business domains Data Warehouse & BI and Organization turn out to be a success factor, and that on the other hand business domains Client & Account Management and Mortgages ended up to be a Bad Practice. One might assume that in the three given companies the best people dealt with Data Warehouse & BI and Organization projects, and that Visual Basic skills were stronger than other skills. And on the opposite, that within the business domains Client & Account Management and Mortgages had to deal with lower skilled resources.

However, it is more likely that Data Warehouse & BI, Organization and Visual Basic project environments on average are less complex than others, and due to that end up in the Good Practice quadrant more often. And that Client & Account Management and Mortgages related project environments on average are more complex than others (e.g. legacy, migration, and dependencies with other departments or domains play a relatively large role here). Benchmarks (e.g. [6]) do confirm this assumption.

6.4 Generalization

Analysis of the performance of the separate companies showed that Company itself was not a distinguishing factor that could be strongly related to either a high percentage of Good Practice or Bad Practice. We assume that the results of our research general-

ize given this finding, in combination with the fact that our research included three different software companies.

7. RELATED WORK

In the following, we discuss the contribution of our study in the context of earlier research. Much has been written on identification of success and failure factors for software engineering, in many cases with reference to software process improvement (SPI), a popular process-based approach to delivering improvements in software products from the 80s [17]. Reel [18] argues that in software, more “advanced” technologies are far less critical to improving practice than five essential factors to managing a successful software project: start on the right foot, maintain momentum, track progress, make smart decisions, and institutionalize post-mortem analyses. Dybå [19] did find strong support for SPI success to be positively associated with business orientation, employee participation, concern for measurement, and exploitation of existing knowledge, and partial support for SPI success to be positively associated with involved leadership and exploration of new knowledge. In another study Dybå [20] showed that small organizations reported that they implement SPI elements as effectively as large organizations, and in turn, achieve high organizational performance. Niazi et al. [21] identified seven success factors for SPI: management support, training, awareness, allocation of resources, staff involvement, experienced staff and defined SPI implementation methodology. Rainer et al. [22] found four SPI success factors: reviews, standards and procedures, training and mentoring, and experienced staff.

In the 90s more advanced process improvement models such as CMMI, and ISO’s SPICE were introduced [23], leading to supplementary research on success and failure factors. Stelzer and Mellis [24] describe ten process oriented factors that affect organizational change in software process improvement initiatives based on the CMM and ISO quality standards. Mahmood et al. [23] inventoried three categories of critical success factors: awareness, organizational, and support. A number of critical barriers were identified (a.o. lack of resources, time pressure, inexperienced staff, organizational politics, and lack of formal methodology). Procaccino et al. [25] found the most important factors for project success to be the presence of a committed sponsor and the level of confidence that the customers and users have in the project manager and development team. Charette [26] names twelve factors why software fails so often, however there is no clear link with the results from our research with regard to these factors.

The common idea in these papers is that success and failure were interconnected with process-based activities: in other words, follow the process and success will come. Looking at the results of our research, a close link with process related research, often based on software projects performed during the last two decennia of the former century, seems absent due to the fact that we did not study this: no clearly process related (in terms of SPI, CMMI, or ISO) factors were present in our list of project factors.

Recent research focusing on the factors that affect success and failure of agile software engineering is more similar. Chow et al. [27] state that, ‘as long as an agile project picks a high-caliber team, practices rigorous agile software engineering techniques and executes a correct agile-style delivery strategy; the project could be likely to be successful’. Three other factors that could be critical to certain success dimensions are a strict agile project management process, an agile-friendly team environment, and a strong customer involvement. Although not stated in the same words a similarity with the results from our research is noticeable here. On

the other hand we did not find evidence that some assumed prerequisites for success of agile projects such as strong executive support, strong sponsor commitment, ready availability of physical agile facility, or agile-appropriate project types, are actually critical factors for success. Misra et al. [28] found nine factors that have statistically significant relationship with success in adopting agile software development practices: customer satisfaction, customer collaboration, customer commitment, decision time, corporate culture, control, personal characteristics, societal culture, and training and learning. No clear link with results from our research is found here. Sutherland et al. [29] studied best practices in a globally distributed Scrum environment, with fixed, experienced agile teams, and comes up with, as the paper states ‘the most productive Java projects ever documented’. Maybe stated differently, however, a match with our findings is obvious here. Otherwise, the idea that agile is always linked to project success is not shared by Estler et al. [30] stating that ‘choosing an agile rather than a structured process does not appear to be a crucial decision for globally distributed projects.’

Our research indicates a relation between agile (Scrum) and economy of scale (project size). Boehm [1] states that ‘the most widely adopted agile method has been XP, whose major technical premise was that its combination of customer collocation, short development increments, simple design, pair programming, refactoring, and continuous integration would flatten the cost-of change-vs.-time curve. However, data reported so far indicate that this flattening does not take place for larger projects.’ Apparently the success of agile does not hold for larger sized projects, something that matches an assumption in our research, since no medium large or large projects were performed based at an agile delivery method. The connection between agile (Scrum) and project size is not clear yet and seems a challenge for future research.

A fascinating gap in related research turns up with relation to the finding from our research that a programming language (Visual Basic) and specific business domains (Data Warehouse & BI, Organization) are significant for project success. It might be surprising that a view at improvement, focusing at benchmarking the outcome of the software process, instead on the software process itself, seems less represented in related work. Jones [31] inventories successful and unsuccessful project technologies, stating that unsuccessful projects are related to a lack of measurement activities, and conversely, successful projects are related to accurate measurement and analysis activities. Jones [6] inventoried a large amount of business domain and programming language specific effects on project performance. In the 90s he performed extensive research on performance aspects related to domain-specific and language-specific software engineering, however since that not many additional research on this area, and especially on the effects of agile delivery models, seem to be performed. Premrai et al. [32] investigated how software project productivity had changed over time, finding that an improving trend was measured, however less marked since 1990. The trend varied over companies and business sectors, a finding that matches the result of our research with regard to differentiation over business domains.

8. CONCLUSIONS AND FUTURE WORK

8.1 Impact

We found 7 Success Factors for software projects: (1) Steady Heartbeat, (2) Fixed, experienced team, (3) Agile (Scrum), (4) Release-based (one application), (5) Business Domain Data Warehouse & BI, (6) Business Domain Organization, and (7) Programming Language Visual Basic.

We found 9 Failure Factors for software projects: (1) Rules & Regulations driven, (2) Dependencies with other systems, (3) Technology driven, (4) Once-only project, (5) Security, (6) Many team changes, inexperienced team, (7) Business Domain Mortgages, (8) Business Domain Client & Account Management, and (9) Migration.

Based on the findings we identify the following guidelines for practice when aiming for a Good Practice Project Portfolio:

1. Avoid Bad Practice by steering at limitation of interdependencies between projects and systems, stay away from unnecessary team changes, and build teams where possible with experienced team-members.
2. Create Good Practice by actively steering on organizing software development in a release-based way, set up fixed teams with experienced team-members, implement a steady heartbeat, and go for an agile (Scrum) delivery approach where applicable.
3. When setting up a once-only project within a software project portfolio, pay special attention to standardization and limitation of procedures to smoothen the projects progress, re-use of knowledge of other once-only projects (Lessons Learned), and implementation of a learning cycle.
4. Implement a long- and medium term portfolio strategy, including a learning capability, to avoid Bad Practice by limitation (where possible) of projects that are characterized as technology driven, rules & regulations driven, migration of data, and security issues.

8.2 Future Work

An interesting side-effect of the observation is that nowadays software companies, in an attempt to become more agile, tend to opt for small releases. Yet maybe active steering on economy of scale can be an equally effective – or even more effective – improvement strategy for software companies. We did not study this side-effect; however future research on the background of economy of scale versus agile development methods might help software companies to find an optimum on project size when building a portfolio. Another aspect that was not covered in our research, yet might help companies to achieve continuous improvement, is whether the software companies did really improve their performance over time. During the six year measurement period many improvement actions were undertaken, yet how successful were these actions in the end? Future research (on our dataset) might reveal this.

9. ACKNOWLEDGMENTS

We thank Hans Jägers, professor emeritus from the University of Amsterdam, Rob de Munnik (QSM Europe), Bart Griffioen, Georgios Gousios, Steven Raemaekers, and all other reviewers for their valuable contributions.

10. REFERENCES

- [1] B. Boehm, “A View of 20th and 21st Century Software Engineering,” in *IEEE International Conference on Software Engineering (ICSE)*, Shanghai, China, 2006.
- [2] M. Tihinen, P. Parviainen, T. Suomalainen and K. Karhu, “ABB Experiences of Boosting Controlling and Monitoring Activities in Collaborative Production,” in *6th IEEE International Conference on Global Software Engineering (ICGSE)*, Helsinki, 2011.

- [3] C. Jones, "Software Cost Estimating Methods for Large Projects," *Crosstalk - The Journal of Defense Software Engineering*, pp. 8-12, 2005.
- [4] K. Moløkken and M. Jørgensen, "A Review of Surveys on Software Effort Estimation," *IEEE - Proceedings of ISESE - International Symposium on Empirical Software Engineering*, pp. 223-230, 2003.
- [5] A. Dagnino, "Estimating Software-Intensive Projects in the Absence of Historical Data," in *IEEE - International Conference on Software Engineering (ICSE)*, San Francisco, CA, USA, 2013.
- [6] C. Jones, *Software, Assessments, Benchmarks, and Best Practices*, New York: Addison Wesley Longman, 2000.
- [7] IFPUG, IFPUG FSM Method: ISO/IEC 20926 - Software and systems engineering – Software measurement – IFPUG functional size measurement method, New York: International Function Point User Group (IFPUG), 2009.
- [8] NESMA, NESMA functional size measurement method conform ISO/IEC 24570, version 2.2, Netherlands Software Measurement User Association (NESMA), 2004.
- [9] K. R. Lindberg, "Software developer perceptions about software project failure: a case study," *Elsevier - The Journal of Systems and Software*, vol. 49, pp. 177-192, 1999.
- [10] CMMI Product Team, CMMI for Development, Version 1.3, Hanscom: Carnegie Mellon, 2010.
- [11] S. Kan, *Metrics and Models in Software Quality Engineering*, Addison Wesley Longman, Reading, Massachusetts, 1995.
- [12] H. Huijgens, R. v. Solingen and A. v. Deursen, "How To Build a Good Practice Software Project Portfolio - Technical Report TUD-SERG-2013-019," Delft University of Technology, Delft, The Netherlands, 2013.
- [13] H. Huijgens and R. v. Solingen, "Measurement of Best-in-Class Software Releases," in *23rd International Workshop on Software Measurement and the Eighth International Conference on Software Process and Product Measurement (IWSM-MENSURA)*, Ankara, Turkey, 2013.
- [14] B. W. Boehm and K. J. Sullivan, "Software Economics: A Roadmap," in *ACM Future of Software Engineering*, Limerick, Ireland, 2000.
- [15] L. Angelis, I. Stamelos and M. Morisio, "Building a Software Cost Estimation Model Based on Categorical Data," in *IEEE Seventh International Software Metrics Symposium (METRICS)*, London, UK, 2001.
- [16] C. F. Kemerer, "An Empirical Validation of Software Cost Estimation Models," *Communications of the ACM*, vol. 30, no. 5, pp. 416-429, 1987.
- [17] T. Hall, A. Rainer and N. Baddoo, "Implementing Software Process Improvement: An Empirical Study," *Software Process Improvement and Practice*, vol. 7, pp. 3-15, 2002.
- [18] J. Reel, "Critical Success Factors in Software Projects," *IEEE Software*, Vols. May-June, pp. 18-23, 1999.
- [19] T. Dyba, "An Empirical Investigation of the Key Factors for Success in Software Process Improvement," *IEEE Transactions on Software Engineering*, vol. 31, no. 5, pp. 410-424, 2005.
- [20] T. Dybå, "Factors of Software Process Improvement Success in Small and Large Organizations: an Empirical Study in the Scandinavian Context," in *ESEC/FSE*, Helsinki, Finland, 2003.
- [21] M. Niazi, D. Wilson and D. Zowgh, "Critical Success Factors for Software Process Improvement Implementation: An Empirical Study," *Software Process Improvement and Practice*, vol. 11, pp. 193-211, 2006.
- [22] A. Rainer and T. Hall, "Key success factors for implementing software process improvement: a maturity-based analysis," *Journal of Systems and Software*, vol. 62, no. 2, pp. 71-84, 2002.
- [23] M. Niazi, D. Wilson and D. Zowghi, "A maturity model for the implementation of software process improvement: an empirical study," *The Journal of Systems and Software*, 2003.
- [24] D. Stelzer and W. Mellis, "Success Factors of Organizational Change in Software Process Improvement," *Software Process Improvement and Practice*, vol. 4, pp. 227-250, 1998.
- [25] J. Procaccino, J. Verner and S. Overmyer, "Case Study: Factors for Early Prediction of Software Success & Failure," *Elsevier - Information and Software Technology*, vol. 44, no. 1, pp. 53-62, 2002.
- [26] R. N. Charette, "Why Software Fails," *IEEE Computing / Software*, vol. September, pp. 1-9, 2005.
- [27] T. Chow and D.-B. Cao, "A survey study of critical success factors in agile software projects," *The Journal of Systems and Software*, vol. 81, pp. 961-971, 2008.
- [28] S. C. Misra, V. Kumar and U. Kumar, "Identifying some important success factors in adopting agile software development practices," *The Journal of Systems and Software*, vol. 82, pp. 1869-1890, 2009.
- [29] J. Sutherland, A. Viktorov, J. Blount and N. Puntikov, "Distributed Scrum: Agile Project Management with Outsourced Development Teams," in *40th International Conference on System Sciences*, Hawaii, 2007.
- [30] H. C. Estler, M. Nordio, C. A. Furia, B. Meyer and J. Scheider, "Agile vs. Structured Distributed Software Development: A Case Study," in *IEEE Seventh International Conference on Global Software Engineering (ICGSE)*, Porto Alegre, Brasil, 2012.
- [31] C. Jones, "Patterns of large software systems: failure and success," *Computer*, pp. 86-87, 1995.
- [32] R. Premrai, M. Shepperd, B. Kitchenham and P. Forselius, "An Empirical Analysis of Software Productivity Over Time," in *IEEE International Symposium Software Metrics*, Como, Italy, 2005.